

ECE220 Midterm 2 Review

Author: Eisa Kamran, David Zheng, Kyle Chung

September 28, 2026

1 Recursion and Arrays

- (a) **All Possible Paths:** Given a 2D square grid with obstacles, write a recursive function to count the total number of paths from the top-left cell to the bottom-right cell. You can only move right or down, and you can only move into squares on the grid marked as '0', and must not leave the bounds of the grid. N is provided as the length of the side of the square grid.

A function skeleton for `num_paths()` has been provided for you, fill in the blanks. You must also fill in the blanks in the `main()` function to the first call of `num_paths()`.

```
1  int example_grid[N * N] =
2  {
3  0, 0, 0, 0,
4  0, 1, 0, 0,
5  0, 1, 1, 0,
6  0, 0, 0, 0
7  };
8
9  int main() {
10     answer = num_paths(example_grid, 0, 0 );
11     printf("Num paths = %d", answer );
12     return 0;
13 }
14
15 int num_paths(int array[], int x, int y) {
16
17     int paths = 0;
18     // Reached destination
19     if (x == N-1 && y == N-1) {
20         return 1;
21     }
22     // Out of bounds
23     if (x >= N || y >= N) {
24         return 0;
25     }
26     // Obstacle
27     if (array[x * N + y] == 1) {
28         return 0;
29     }
30
31     // Recursive Cases
32     paths += num_paths(array, x, y+1);
33     paths += num_paths(array, x+1, y);
34
35     return paths;
36 }
```

- (b) **Minesweeper:** Minesweeper is a game where you are presented with a grid and you try not to step on a bomb. We present you a simplified version of minesweeper. When you tap on a cell, there are three possibilities.

- 1) if the cell is surrounded by at least one bomb, the cell is marked with the number of bombs it neighbors.
- 2) The cell itself has a bomb, in which case the game is over.
- 3) If no bombs surround the cell, then all of its neighboring cells are marked by how many bombs neighbor them. Of course, if a neighboring cell has no bombs, the process repeats for that cell. This is where recursion comes into play. For example, given a sample hidden grid and the corresponding sample blank player grid:

Hidden grid:

x		x	x	x
			x	

Player grid:

If you tap on the cell at row 1, column 1, the output player grid should look like

	2			

However, if you tap on the cell at row 2, column 1, the output player grid should look like

1	2	3		
0	0	1		
0	0	1		

A function skeleton for `fillPlayerGrid()` has been provided for you. It takes in 6 parameters:

`toFill`: this is an int array in row-major representing what the player grid should look like after the tap. All cells are initialized to -1, indicating unmarked cells. You are tasked with filling in this array so that it represents the player grid after a tap at a certain cell.

`hiddenGrid`: This is an int array in row-major where a 1 represents a bomb in that cell and a 0 represents no bombs. We have also provided a helper function, `countBombs()`, which takes in a hidden grid and indices `r` and `c` and returns you how many bombs neighbor that cell.

`r`: this is an integer representing the row of the cell tapped. You may assume it is in-bounds.

`c`: this is an integer representing the column of the cell tapped. You may assume it is in-bounds.

`rows`: this is an integer representing the number of rows of the grid.

`columns`: this is an integer representing the number of columns of the grid.

Fill in the blanks for `fillPlayerGrid()`.

```

1      /*
2      toFill — array represented in row-major where all cells are initialized
with -1s.
3      Your job is to fill in the corresponding cells.
4      hiddenGrid — array represented in row-major array with locations of the
bombs
5      r — row index of current cell
6      c — column index of current cell
7      rows — number of rows in the grid
8      columns — number of columns in the grid
9
10     You are given a helper function, countBombs().
11     Given a cell at (r,c), it returns the number of neighboring bombs at that
cell.
12     Function signature:
13
14
15     int countBombs(int * hiddenGrid, int r, int c, int rows, int columns)
16     The parameters are the same as above.
17     */
18
19
20
21     void fillPlayerGrid(int * toFill, int * hiddenGrid, int r, int c, int rows, int
columns)
22     {
23
24         //Base Case: There is a bomb AT the cell
25         if (hiddenGrid[r * columns + c])
26         {
27             return;
28         }
29         //Base Case: The cell has at least one neighboring bomb cell
30         // get number of neighboring bombs using countBombs()
31         int centerBombs = countBombs(hiddenGrid, r, c, rows, columns);
32
33         if (centerBombs != 0) // If number of bombs is not zero
34         {
35             // Set toFill at (r,c) to the number of bombs
36             toFill[r * columns + c] = centerBombs;
37
38             return;
39         }
40
41         // Recursive case: The cell has zero neighboring bomb cells; check its
neighbors
42         //go from the previous row to the next row
43         for (int x = r-1; x <= r + 1; x++)
44         {
45             if ( x >= 0 && x < rows ) // check if we are in row bounds
46             {
47                 // Go from the previous column to the next column
48                 for (int y = c - 1; y <= c + 1 ; y ++ )
49                 {
50                     if ( y >= 0 && y < columns) // check if we are in column
51                     bounds
52                     {
53                         if ( x != r || y != c ) // check if this is not the same
54                         cell at (r,c)
55                         {
56                             // get count of number of neighboring bomb cells for
57                             cell (x,y)
58                             int bombs = countBombs(hiddenGrid, x,y,rows,columns);
59
60                             // If we haven't already updated toFill at (x,y)
61                             if (toFill[x * columns + y] == -1 )

```

```

61         {
62             if ( bombs > 0)
63                 // if the cell at (x,y) has at least one
64                 neighboring bomb
65                 {
66                     toFill[x * columns + y] = bombs;
67                 } else
68                 {
69                     toFill[x * columns + y] = 0;
70                     //Recursively call using x and y
71                     fillPlayerGrid(toFill, hiddenGrid, x, y, rows,
72                                   columns);
73                 }
74             }
75         }
76     }
77 }
78 }
79 }
80

```

2 C to Lovely LC3 Conversion

1.: Gana decides to write a function to calculate his score on STAT 400 exams based on a measure of how much sleep, studying, and food he has. He came up with the code below:

```

1 int score(s, ss, f) {
2     int score;
3     score = ss + f - s;
4     return score;
5 }
6
7 int Gana()
8 {
9     int sleep = 8;
10    int study = 7;
11    int food = 3;
12
13
14    return score(sleep, study, food);
15
16 }
```

Gana() runs as stated and hits the "score" function. Please complete the following function

Caller setup: Please complete the Lc3 code and how the RTS diagram looks like after caller setup. Assume that sleep, study, and food are stored in R1, R2, and R3 respectively. Please ensure to draw where R6 and R5 are after caller setup ends. Assume the values for the variables have already been set in the stack. RTS:

Value	
8	sleep (arg) ← R6
7	study (arg)
3	food (arg)
3	food (local)
7	study (local)
8	sleep (local) ← R5

LC3 code:

; Please load sleep, study, and food into R1, R2, and R3 ; respectively

- LDR R1, R6, #2
- LDR R2, R6, #1
- LDR R3, R6, #0

; Push Score arguments to stack (only update stack ; pointer once)

- ADD R6, R6, #-3
- STR R1, R6, #0
- STR R2, R6, #1
- STR R3, R6, #2

; Call score

- JSR score

Callee setup: After callee setup ends, Please draw what the stack looks like, including updated R6 and R5 values

RTS:

Value	
	score (local) $\leftarrow$ R6, R5
Old R5	dyn link
R7	ret addr
	ret val
8	s (arg)
7	ss (arg)
3	f (arg)
3	food (local)
7	study (local)
8	sleep (local)

LC3 code:

; Set up bookkeeping information and allocate space
; for the local variable (only update stack pointer once)

- **ADD R6, R6, #-4**
- **STR R7, R6, #2**
- **STR R5, R6, #1**
- **ADD R5, R6, #0**

Function Logic:

; Pop out variables into R2, R3, R4
; add up everything, store result into R4
; store R4 into score

- **LDR R2, R6/R5, #4**
- **LDR R3, R6/R5, #5**
- **LDR R4, R6/R5, #6**
- **NOT R2, R2**
- **ADD R2, R2, #1**
- **ADD R4, R4, R2**
- **ADD R4, R4, R3**
- **STR R4, R6, #0**

Callee Teardown: Please write the code for callee teardown, and then fill in the RTS after Callee Teardown has occurred. Make sure to include where R5 and R6 are as well!
RTS:

Value	
2	score (local)
Old R5	dyn link
R7	ret addr
2	ret val $\leftarrow$ R6
8	s (arg)
7	ss (arg)
3	f (arg)
3	food (local)
7	study (local)
8	sleep (local) $\leftarrow$ R5

LC3:

; store score in return value, using R0 as a temp register

- **LDR R0, R6, #0**
- **STR R0, R6, #3**

; restore R7, return, restore CFP, and deallocate space on the stack

- **LDR R7, R6, #2**
- **LDR R5, R6, #1**
- **ADD R6, R6, #3**
- **RET**

Caller Teardown: Complete caller teardown, and show what RTS looks like after Caller teardown completes (make it look like how it was when we started the first question, not full caller teardown)
RTS:

Value	
2	score (local)
Old R5	dyn link
R7	ret addr
2	ret val
8	s (arg)
7	ss (arg)
3	f (arg)
3	food (local) $\leftarrow$ R6
7	study (local)
8	sleep (local) $\leftarrow$ R5

LC3:

; Pop Return value into R3
; finish tearing down stack

- **LDR R3, R6, #0**
- **ADD R6, R6, #4**

3 Conceptual

- (a) **Stack:** When using a stack, when you pop an item off the stack, is it removed from memory?
No. The value is still in memory. Only the top of stack pointer (R6) is moved down (incremented), so that spot is treated as free and gets overwritten by the next push.
- (b) **Pointers:** Assume we declare 4 separate pointer variables in a C program on a 32-bit system as shown below.

```
1   int* david;  
2   char* eisa;  
3   float* kyle;  
4   void* xavier;  
5
```

Which one of these variables takes up the most space in memory?

- a. david
 - b. eisa
 - c. kyle
 - d. xavier
 - e. They are the same size
 - f. They take up no memory space
- e. is correct. All of the variables are pointer types, so they all take up the same size in memory (32 bits each, since this is a 32-bit system). Although it is true that the values they point to take up different sizes in memory, with char* being the smallest (8 bits).
- (c) **Recursion:** What are some potential downsides to a recursive solution? 1) Stack Overflows. A recursive function uses the run time stack for each function call. Depending on the depth of the recursion, this may cause the stack size to grow greater than the amount of memory on the system, resulting in various problems. 2) Performance. Similar to stack overflows, there is a lot of additional overhead in setting up and tearing down stack frames for each function call. This can cause a slow program if the compiler is unable to optimize it.
- (d) **2D Arrays:** Suppose we have a 2d array stored in column major order as.

123	93	0	76	67
921	82	10	4	5
42	13	43	9	65
100	2	54	10	32

How do you access the array to retrieve the element '9'? Assume the array is declared as array[4][5]. Write two answers using both 1d and 2d array notation.

2D: array[2][3] (row 2, column 3), 1D (column-major): array[3 * 4 + 2], which is array[14];

- (e) **Passing Pointers:** Explain the importance of having the parameters of this function be pointers.

```
1 void swap(int* a, int* b)  
2 {  
3     int temp;  
4     temp = *a;  
5     *a = *b;  
6     *b = temp;  
7 }
```

Having parameters as pointers allow the data pointed to to change from within the function. If the variables were passed in as just their values, only the run time stack variables would be edited and the new values would not be saved. Note how this function doesn't return anything, but is still useful.

- (f) **Arrays:** in C, Arrays in function arguments / parameters are...
- Pass by reference
 - Pass by value
 - Either/or

The answer is a. Arrays in C always decay to a pointer to their first element when passed in functions. This means the entire array isn't copied onto the run time stack every time it is passed into a function. Because arrays represent contiguous regions in memory, just having a pointer to the first element is enough to access any element in the array. Note: The only way to pass an entire array is to wrap it in a struct.

- (g) **Rambunctious Recursion:** When writing a recursive algorithm, what is the goal of each recursive step? (Hint: The base case represents the simplest form of the problem)

The goal of each recursive step is to make the problem slightly simpler. An example is factorial, you calculate $\text{number} * \text{factorial}(\text{number}-1)$. By decrementing the number each time, we make the problem easier, until we get to the base case.

- (h) **Sorting algorithms:** Aaron writes the following algorithm. Which sorting algorithm did he use?

```

1 void sort(int arr[], int n) {
2     int i, j;
3     for (i = 0; i < n - 1; i++)
4         for (j = 0; j < n - i - 1; j++)
5             if (arr[j] > arr[j+1])
6                 swap(&arr[j], &arr[j + 1]);
7 }
```

Bubblesort

- (i) **Midpoint:** What is wrong with Lucas's recursive midpoint function?

```

1 find_midpoint(int a, int b) {
2     if (a == b) { return a; }
3     else { return find_midpoint(a+1, b-1); }
4 }
```

There are two main things wrong. The first, is that there is no return type for this function, we need to make this return a float. The second, is that we need to deal with numbers that are odd-length apart, for example 4 and 7. To do this we can add: `else if (a+1 == b) return ((a+b)/2.0);` as another condition to check.

- (j) **Recursive Reversal:** What is the output of this program? If there is an error in ReverseArray, identify the line and fix it? (Hint: it might be nice and helpful to print every step of Reverse Array)

```

1 void ReverseArray(int array[], int size) {
2     int start = 0;
3     int end = size - 1;
4     int temp;
5
6     if (start < end) {
7         // Swap First and Last
8         temp = array[start];
9         array[start] = array[end];
10        array[end] = temp;
11
12        ReverseArray(array, size-1);
13    }
14 }
15 int main(){
16     int array[5], i;
17     for (i = 0; i<5; i++){
18         array[i] = i;
19     }
20     ReverseArray(array, 5);
21     printf("Reversed Array: ");
22
23     for (i = 0; i<5; i++){
24         printf("%d ", array[i]);
25     }
26     printf("\n");
27     return 0;
28 }

```

The program is basically just flipping the first and last elements, and then flipping the first and second to last and so on. So in the end, the only thing that actually changes is the first element goes to the back and the rest is shifted one spot toward the front. The output is: Reversed Array: 1 2 3 4 0. To fix this we need to change the recursive call on line 12 to ReverseArray(array+1, size-2)

- (k) **Arrays on stack:** Rahul initializes an array and calls a function "print" on it. Please write what the RTS looks like after Callee setup.

```

1 int main{
2     int agi = 1;
3     char arr[5] = {'R', 'A', 'H', 'U', 'L'};
4     print(arr, agi);
5 }

```

Value	
local variable	← R5, R6
caller frame pointer	
return address	R7
return value	
pointer to arr[0]	arr (arg)
1	agi (arg)
'R'	arr[0] (local)
'A'	arr[1] (local)
'H'	arr[2] (local)
'U'	arr[3] (local)
'L'	arr[4] (local)
1	agi(local)

(1) **Alternative Indexing:** Fill in the blanks to make the two arrays have identical values.

```
1 int array1[4][2];
2 int array2[8];
3 int i, j;
4 for (i = 0; i < 2; i++) {
5     for (j = 0; j < 4; j++) {
6         array1[ j ][ i ] = i + j;
7         array2[ 2 * j + i ] = i + j;
8     }
9 }
```

4 Extra Problems

- (a) **Difficult Student Sort:** Fill In the blanks to find the student with the highest GPA and store a pointer to them in `best_student`

```

1  typedef struct StudentStruct {
2      int UIN;
3      float GPA;
4  } Student;
5
6  int main () {
7      Student all_students[5];
8      // Load data into all students:
9      load_students(all_students, 5);
10     // Find the student with the highest GPA:
11     Student* best_student = &(all_students[0]) ;
12     find_best(all_students, 5, &best_student );
13     printf("Best GPA:%f\n", best_student->GPA);
14 }
15
16 void find_best(Student* all, int num_students, Student** best) {
17     for (int i = 0; i < num_students; i++) {
18         if (all[i].GPA > (*best)->GPA) {
19             *best = &(all[i]);
20         }
21     }
22 }

```

- (b) **C to LC3:** Convert the following C function into LC3 using Callee Setup and Teardown.

```

1  int foo(int a, int b) {
2      int x;
3      x = a + b;
4      return x;
5  }

```

```

ADD R6, R6, #-1           ; Allocate spot for return value
ADD R6, R6, #-1
STR R7, R6, #0            ; Push R7 (Return Address)
ADD R6, R6, #-1
STR R5, R6, #0            ; Store R5 (Caller's Frame Pointer)
ADD R5, R6, #-1           ; Set frame pointer for Callee
ADD R6, R6, #-1           ; Allocate memory for local variables (x)
LDR R1, R5, #4             ; Load a
LDR R2, R5, #5             ; Load b
ADD R0, R1, R2
STR R0, R5, #0            ; Store result into x
STR R0, R5, #3            ; Store result into return value slot
ADD R6, R5, #1            ; Pop local variables
LDR R5, R6, #0            ; Pop Frame Pointer
ADD R6, R6, #1
LDR R7, R6, #0            ; Pop Return Address
ADD R6, R6, #1
RET

```